

Here's a format for a lab report based on the code you provided for the **Midpoint Circle Drawing Algorithm**:

Title: Implementation of Midpoint Circle Drawing Algorithm Using OpenGL

Objective:

The primary objective of this experiment is to implement the **Midpoint Circle Drawing Algorithm** using OpenGL. This algorithm is used for efficiently drawing circles by determining the next pixel based on the previous one using integer calculations, minimizing the use of floating-point arithmetic. The algorithm will demonstrate the fundamentals of drawing circles in a raster display by leveraging symmetry properties.

Theory:

The **Midpoint Circle Drawing Algorithm** is a rasterization algorithm used in computer graphics to render a circle. It exploits the symmetry of a circle to reduce the amount of computation required by plotting only one-eighth of the circle and then reflecting the points in other octants.

The decision parameter p_k is used to choose the next pixel based on the midpoint between two possible pixel candidates. Depending on the value of p_k , we either plot directly adjacent pixels or move diagonally.

Key equations:

- Initial decision parameter:
 $[p_0 = 1 - r]$ where r is the radius of the circle.
 - If the decision parameter is negative, the next pixel lies directly horizontal to the current pixel.
 - If the decision parameter is positive, the next pixel lies diagonally.
-

Algorithm:

1. **Input:** Center of the circle (x_c, y_c) and radius (r) .
 2. **Initialize:** Set the starting point of the circle at $(x, y) = (0, r)$, and calculate the initial decision parameter as $(p_0 = 1 - r)$.
 3. **Loop:** Continue iterating while $(x \leq y)$:
 - Plot the current point and its reflections in all 8 octants of the circle.
 - Update the decision parameter (p_k) :
 - If $(p_k < 0)$, update $(p_{k+1} = p_k + 2x + 1)$.
 - Else, update $(p_{k+1} = p_k + 2x - 2y + 1)$, and decrement (y) .
 - Increment (x) .
 4. **Output:** Display the circle on the screen.
-

Discussion:

1. Symmetry Utilization:

The algorithm utilizes the symmetry of a circle to reduce computational overhead. By calculating points for just one octant and reflecting these in other octants, it minimizes the number of calculations.

2. Decision Parameter:

The decision parameter p_k helps determine the next pixel to plot. This reduces the use of square roots and floating-point arithmetic, making the algorithm efficient.

3. Challenges Encountered:

- **Floating Point Rounding:** Handling floating-point precision in OpenGL functions, though mostly handled by integer arithmetic here, can sometimes lead to minor visual inconsistencies.
- **OpenGL Setup:** Proper initialization and setting up the viewing window and coordinate system were crucial to ensure the circle is rendered correctly.
- **Coordinate System Adjustments:** Since OpenGL uses a Cartesian coordinate system, transformations are necessary to map pixels correctly, especially for non-centered circles.